

Encoding HTN Heuristics in PDDL Planning Instances - Extended Abstract

Christoph Mies and Joachim Hertzberg

Osnabrück University
Albrechtstraße 28, 49076 Osnabrück, Germany
{cmies, joachim.hertzberg}@uos.de

1 Contribution and Related Work

The paper sketches a transformation process that allows *Hierarchical Task Network (HTN)* based domain-dependent planning heuristics to be encoded into *Planning Domain Definition Language (PDDL)* [1] planning instances. SHOP2 [2] is chosen as a representative of the many different HTN planning approaches in the literature, but the paper discusses modifications of the transformation applicable to other HTN based planning approaches, too. The evaluation, based on five state-of-the art PDDL planners from the public domain, shows that incorporating HTN heuristics accelerates the planning process in terms of computation time and improves the quality of the resulting plans.

Automated Planning is a well-known and powerful decision making technique. Planning systems are applied to decision problems, called planning instances, from various application areas. They exploit abstract domain independent search heuristics for problem solving. In this paper, we encode HTN based domain-dependent planning heuristics into PDDL planning instances.

PDDL was designed to be a standard language for planning instances. Its motivation was to make different planning systems comparable and planning instances re-usable to create standard benchmarks for planning systems [1]. The maxim behind the language specification was to model “physics, not advice” [1, p. 4]. Additionally, the language was designed to be neutral, i.e. “it doesn’t favor any particular planning system” [1, p. 4]. These prerequisites set up barriers for planning systems exploiting domain-dependent search heuristics. In particular, PDDL does not fulfill the input requirements of HTN based planning approaches. For the first *International Planning Competition (IPC)*, McDermott noticed that “researchers with hierarchical planners lost interest rapidly as it became clear how great the distance was between PDDL and the kind of input their planners expected” [1, p. 17].

The most important advantages of considering domain-dependent planning heuristics during planning are these: Firstly, domain experts are usually able to provide heuristics that enable *fast generation* of plans with *high quality*. Additionally, the resulting plans usually *match the expectations* of the domain designer, since the plan generation can be *explained* with respect to the given HTNs, i.e. the designer can *trace* the task decomposition and adapt it if required.

The contribution of this paper is the sketch of a translation of a planning instance of one well-known representative of the HTN planning approaches, SHOP2, to PDDL. As a result of the introduced transformation, we encode the HTN based domain-dependent planning heuristics of SHOP2 planning instances into PDDL planning instances, which forces the PDDL planner to follow SHOP2’s task decomposition strategy. Thus, we close the gap between the input needed for SHOP2 and PDDL’s expressiveness. Due to lack of space, we concentrate on the management of open SHOP2 tasks in the PDDL planning instances. This management is flexible enough to support the encoding of other HTN based planning approaches, too.

The idea to transform a planning formalism incorporating control knowledge to domain-independent formalisms is not new. Fritz et al [3] provide a translation from CONGOLOG, a well-known situation calculus based planning formalism, to PDDL 2.1 [4]. The concept of threads employed in the transformation in this paper is inspired by their approach. In the definition of the transformation, we focused on the generation of efficiently solvable planning instances. Since Fritz’s transformation handles CONGOLOG, a more expressive formalism, we expect their generated planning instances to be not as efficiently solvable as ours.

Alford et al [5] provide a transformation from the restricted HTN variant *Total-Order Simple Task Network Planning* [6, ch. 11.3] to PDDL. Our introduced transformation sketch can be modified to support different HTN based planning approaches, especially this one.

Another way to incorporate HTN heuristics into planning is *hybrid planning*. An exemplary approach is introduced by Schattenberg et al [7], who combine HTN based domain-dependent heuristics with partial order causal link (POCL) planning. They abstract the planning operators: As in most planning approaches, an operator has a precondition as well as an effect to model when it is executable and how the world is changed due to its execution. But in addition, it can have partial plans as operator decomposition.

2 Transformation Sketch

Due to lack of space, we only sketch the transformation of planning instances containing HTN definitions to PDDL instances roughly. We have chosen SHOP2 as an exemplary HTN planning formalism.

Each SHOP2 language entity is transformed to a PDDL construct. Basically, SHOP2 tasks and task networks are encoded by PDDL predicates; operators and methods by PDDL actions. Most importantly, the PDDL planning instance must enforce the transformed planning system to follow the SHOP2 task decomposition strategy, which is encoded into the PDDL planning instances.

The key concept of the transformation is the book-keeping of *open tasks* during planning, i.e. tasks that have not yet been accomplished in the current world state. We perform the marking with two types of technical objects that we insert into the PDDL planning instance: **thread** and **stackitem**. When several tasks have to be solved sequentially, they are bound to the same thread object.

If not, the tasks are bound to several thread objects. Each thread has its own stack, which ensures the correct sequential order of its tasks. PDDL requires a finite number of objects in the domain and does not allow numerics as arguments for actions [4, p. 68]. Thus, we have to provide enough thread and stack item objects in the PDDL planning instance in advance. This restriction regards the number of open tasks that can be represented in one PDDL state. The two related approaches [3,5] face similar restrictions. Both use a finite number of objects representing a finite counter.

Now, how many thread and stack item objects are needed in a PDDL planning instance to solve the problem? Unfortunately, the answer is not simple and requires an analysis of the SHOP2 planning instance at hand. To determine the required number of thread and stack item objects, we have to investigate all possible applications of the hierarchical task networks. In terms of SHOP2, we have to determine the possible call hierarchies of the given methods. The number of available threads in a planning instance denotes the number of open tasks that can be solved in parallel and the number of stack items limits the number of open tasks that can be assigned to each thread. Note that the number of required marking objects is not proportional to the resulting plan length, since long plans can be generated by short recursive call chains of methods.

From a logical point of view, the introduction of two object types, thread and stack item, is not needed. The thread type would suffice, since parallel as well as a sequential task relations can be expressed by explicit called-by relations. The management of the thread objects is complex, since we have to ensure that whenever a new thread is chosen by the PDDL planning system, exactly one thread is chosen. The management of stack items is simple. It is possible to employ a stack data structure, since “SHOP2 generates the steps of each plan in the same order that those steps will later be executed” [2, p. 379].

To adapt the introduced transformation to other HTN planning approaches, the marking has to be modified. For example, we consider the HTN definition *Simple Task Network (STN)* [6, ch. 11.2]. The method definition is similar to that of SHOP2, but only allows one single precondition and one decomposition. STNs are used by Alford et al [5] (cf. Section 1) in combination with the planning algorithm *Total-Order STN Planning* [6, ch. 11.3]. This algorithm also generates the plan steps in their final execution order as SHOP2 does. Additionally, the considered task networks are totally ordered. When transforming planning instances of this formalism, our transformation needs exactly one thread and several stack items. Planning instances of the planning algorithm *Partial-Order STN Planning* [6, ch. 11.4] can also be transformed by our approach with minimal modifications.

3 Conclusion

We have sketched an approach to encode HTNs containing domain-dependent planning heuristics in PDDL planning instances. Of the many different variants of hierarchical task networks in the literature, we have chosen SHOP2 as rep-

representative HTN based planning formalism. Encoding domain-dependent HTNs containing planning heuristics to a PDDL is advantageous, because this encoding can speed up the planning process, it can improve the quality of the resulting plans and it may raise the acceptance of human domain experts.

We performed a first evaluation with five different PDDL planning systems (PROBE, SGPLAN, LAMA-2011, CBP and CBP2) being involved¹. They have been applied to the transformed planning instances and, as comparison, to pure PDDL instances without HTN search heuristics. Four of five PDDL systems solve the enriched instances faster, with improved quality or even larger planning instances when solving the transformed planning instance.

The two related approaches introduced in Section 1 [3,5] suffer from the same restriction as our transformation does: PDDL requires that all available (finitely many) objects are defined in the planning instance. Our transformation is, amongst others, able to handle the HTN based planning algorithms transformed by Alford et al [5]. GOLOG, the variant handled by Fritz et al [3], is a very rich formalism. Thus, we expect our transformation to result in more efficiently solvable PDDL planning instances.

In future work, we will apply the transformation to other domains from the literature for further experiments. Then, the cooperation of pure PDDL parts and encoded domain-dependent heuristics within the same PDDL planning instance is of interest. If there are no powerful HTN heuristics for parts of the application domain, the PDDL planning system should solve these parts. Hybrid planning, e.g. [7], exploits such a co-existence of HTN and non-HTN planning and profits from the strengths of both formalisms.

References

1. D. McDermott. The 1998 AI Planning Systems Competition. *AI Magazine*, 21(2):35–55, Summer 2000.
2. D. Nau, T.-C. Au, O. Ilghami, U. Kuter, W. Murdock, D. Wu, and F. Yaman. SHOP2: An HTN Planning System. *JAIR*, 20:379–404, 2003.
3. C. Fritz, J.A. Baier, and S.A. McIlraith. ConGolog, Sin Trans: Compiling ConGolog into Basic Action Theories for Planning and Beyond. In *KR-2008*, pages 600–610, Sydney, Australia, September 16–19 2008.
4. M. Fox and D. Long. PDDL2.1: An Extension to PDDL for Expressing Temporal Planning Domains. *JAIR*, 20:51–124, 2003.
5. R. Alford, U. Kuter, and D. Nau. Translating HTNs to PDDL: A Small Amount of Domain Knowledge Can Go a Long Way. In *IJCAI-2009*, pages 1629–1634, San Francisco, 2009. Morgan Kaufmann Publishers Inc.
6. M. Ghallab, D. Nau, and P. Traverso. *Automated Planning: Theory and Praxis*. Morgan Kaufmann Publishers, Boston, MA, USA, May 2004.
7. B. Schattenberg, A. Weigl, and S. Biundo. Hybrid planning using flexible strategies. In *KI 2005*, volume 3698 of *LNAI*, pages 258–272. Springer, September 2005.

¹ See <http://www.inf.uos.de/kbs/htn2pddl.html> for input files, domain description and evaluation results etc.